

Autonomous Agents for the Deliveroo.js Environment

Autonomous Software Agents - Final Project Report

University of Trento,

Riderless

Abstract

This report describes the design and implementation of an autonomous agent for the **Deliveroo.js** game environment, a grid-world simulation in which agents compete to pick up parcels and deliver them to designated drop zones. The agent follows a *Belief-Desire-Intention* (BDI) architecture written in TypeScript. Decisions such as: which parcel to collect, where to deliver, and where to explore—are driven by a utility function that accounts for parcel reward, decay rate, distance heuristics, and competitive pressure from other agents. Path-finding is handled by a BFS-based planner that can optionally be upgraded to an optimal PDDL solver. A secondary LLM interface allows the agent to interpret natural-language commands, translating them into BDI desires, and to communicate with another agent to cooperate effectively and maximize points.

1 Introduction

1.1 Context and Motivation

Deliveroo.js is a browser-based, multi-agent grid world developed at the University of Trento for the Autonomous Software Agents course. Agents connect to a server over a WebSocket API, receive perceptual updates (parcels, tiles, other agents), and must act continuously to maximize their score. Additionally, the agent can process textual instructions to earn extra points, balancing these requests with its primary tasks whenever possible. This setup provides an ideal testbed to explore how symbolic planning (PDDL) and LLM-based reasoning can complement a classical BDI loop.

1.2 Project Goals

1. **BDI Agent Architecture (Challenge 1):** Design and implement a deliberative agent capable of sensing the environment, dynamically revising beliefs, and managing intentions to collect and deliver parcels efficiently.
2. **Symbolic Planning Integration:** Integrate an asynchronous PDDL planner to support optimal path-finding and navigate complex grid topologies.
3. **LLM Natural-Language Interface (Challenge 2):** Develop an LLM pipeline to handle atomic operator requests, dynamically adapt individual game strategies, and parse complex operational constraints.
4. **Multi-Agent Coordination:** Implement a collaborative strategy allowing the LLM-enhanced agent, the core BDI agent, and the external mission-agent to communicate,

exchange environment data, and coordinate actions toward shared goals.

2 System Architecture

2.1 Overview

The system operates on a reactive sense-deliberate-act cycle, where each server-side sensing event drives a new iteration of the BDI loop. Figure 1 sketches the high-level data flow.

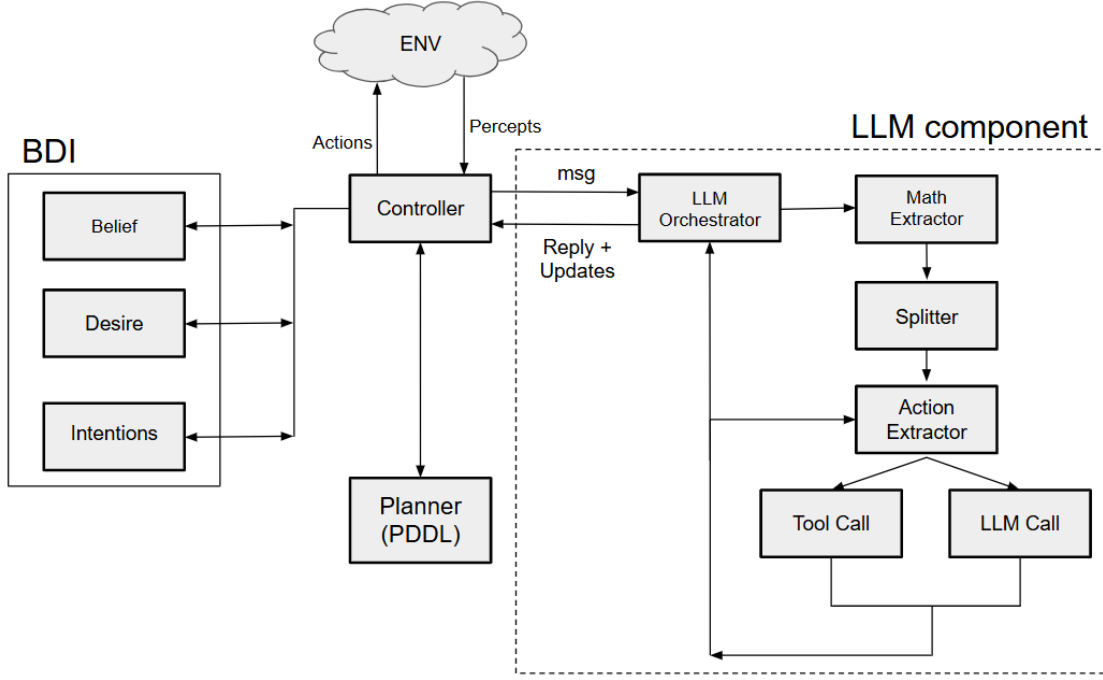


Figure 1: Agent Architecture

The system comprises three main subsystems:

- **BDI**: maintains world beliefs, generates desires, and commits to intentions.
- **Planning Component (PDDL)**: computes movement paths.
- **LLM Component**: interprets natural-language instructions and leverages a dedicated suite of **deterministic tools** (e.g., for mathematical evaluations) to inject updates into the BDI loop.

The entry point `main.ts` wires all components together, handles the WebSocket events, and runs the main BDI step function.

2.2 BDI Agent Model

The BDI layer is split across three files.

Beliefs (BDI/Belief.ts). The belief layer implements the agent’s belief update step, maintaining an internal representation of the environment under partial observability. During initialization, the main socket handlers configure the agent parameters, build the

static map, and initialize the agent’s own position. After this setup, each sensing event updates the dynamic part of the world state, including parcels, crates, and other agents.

Beliefs are stored inside the `World` object as coordinate-indexed maps. When new observations arrive, **beliefs** are **updated**. Entities that are temporarily outside the observation range are not removed immediately; instead, they are kept for a short time using timestamp-based lifespans. This provides the agent with limited short-term memory, making its behavior more robust to partial observability.

The belief update also enriches the raw observations with task-relevant information. For example, opponent positions can be tracked over time to infer movement direction or stationary behavior, while crates positions are refreshed when visible and later used for obstacle modeling. In parallel, the main loop keeps an auxiliary state such as the list of carried parcels by the agent.

Belief revision allows the agent to avoid temporarily blocked cells even when no obstacle is explicitly observed. This accounts for dynamic obstructions, such as agents moving into a cell between sensing cycles (which are then corrected at the next `onSensing` event), as well as cells flagged as blocked via LLM-interpreted textual instructions.

Overall, the belief state is therefore not limited to the latest sensor reading. It combines persistent map information, short-lived memories of dynamic entities, and auxiliary runtime state needed by the BDI loop to select and execute intentions.

Desires (BDI/Desire.ts). A Desire is represented as a tuple $\langle \text{type}, x, y, u \rangle$, where $u \in \mathbb{R}$ denotes its utility score. The agent generates three internal desire types:

- **go_pickup**: generated for each visible parcel that is not currently being carried. Its utility 2.5.1 depends on the parcel reward, the distance required to reach it, and the distance from the parcel to the nearest delivery zone. The score is also reduced when opponents are close to the pickup location, using a competitive penalty based on the number of nearby agents.
- **go_delivery**: generated when the agent is carrying at least one parcel. Its utility 2.5.2 estimates the remaining reward obtainable by reaching the closest free delivery tile, subtracting the expected decay cost of the trip. Delivery tiles occupied by stationary opponents are penalized.
- **explore**: generated as a fallback behavior to guide the agent toward high-density spawn regions that have not been visited recently. Its score combines recency, BFS distance, spawn-cluster density, and opponent pressure 2.5.3. Multiple exploration candidates are generated so that the agent can switch to an alternative target if one path becomes unreachable.

A fourth desire type, **go_to**, is injected exclusively by the LLM component, as described in Section 2.4. New atomic LLM tasks have to be inserted at this level of the architecture.

Intentions (BDI/Intentions.ts). The `reviseIntention` function implements the agent’s **intention revision** policy, which is executed at every BDI step. Based on the updated beliefs and a list of desires sorted by utility, this function determines whether the agent should maintain its current commitment or switch to a better candidate.

The control flow handles two main scenarios. If no intention is currently active, the agent automatically commits to the highest-utility desire. Conversely, if an intention is already active, its ongoing *validity* is verified against the current world state. Specifically, delivery intentions remain valid only while the agent is carrying at least one parcel, whereas pickup intentions are kept only as long as the target parcel is still available on the map. Exploration intentions, on the other hand, are always considered valid and act as a fallback behavior when no other tasks can be performed. If the active intention fails its respective validity check, it is immediately discarded and replaced by the best available desire. Intentions can be **blocked** after a fixed number of failures and the agent automatically switch intention to the next best one.

The BDI intention system is also extended to accommodate **LLM-injected missions**. Missions extracted from natural-language instructions — such as navigation goals (`go_to`), forced delivery targets (`go_delivery`), synchronized wait requests (`wait_row`), or rendezvous points — are queued in a dedicated ordered list (`llmPendingMissions`) and promoted one at a time to an active slot (`llmActiveMission`). These missions carry a high default utility value so that they preempt lower-priority autonomous desires. Stack-constraint rules and delivery-bonus modifiers, by contrast, do not generate new intentions: they instead parametrize the utility computation of existing desire types, allowing external instructions to reshape the agent’s preference ordering without bypassing the normal revision cycle. Upon completion, the active LLM mission is cleared and the agent seamlessly reverts to autonomous behavior.

2.3 Planning Component

The planning component enables the agent to compute a path toward a target tile. To evaluate the performance and efficiency of the planning subsystem, we implement a Breadth-First Search (BFS) algorithm as a baseline for shortest-path calculation and compare it against a classical automated planning approach using PDDL.

The PDDL-based module leverages the *Fast Downward* planner, configuring it to use the A^* search algorithm to determine the optimal path. Within this formalization, the planning domain defines the available actions as the movement capabilities of the agent, while the state space represents the grid tiles of the environment.

Furthermore, the PDDL domain is designed to support crate manipulation in single-agent scenarios, enabling the agent to plan crate movements to navigate more complex environments. However, crate handling in cooperative multi-agent settings is not currently supported.

2.4 LLM Component

The LLM component processes textual user inputs and extracts the structured information required to update the BDI agent’s state. For this purpose, we selected the *Llama-3.3-70B* model due to its advanced reasoning capabilities. To optimize performance and context window efficiency, we condition the model’s short-term memory using concise system prompts paired with tailored *few-shot examples* for each specific task.

The LLM component is designed to handle and execute a variety of tasks, which can be categorized into three levels of complexity:

- **Atomic Special Missions (Level 1):** Simple, self-contained tasks that either produce a direct reply or inject a single transient update into the BDI loop. These include mathematical computations (`calculate`), time zone queries (`get_current_time`), position reports (`get_my_position`), common-knowledge questions (`common_knowledge`), and tile-level navigation or avoidance goals (`generate_desire`).
- **Intermediate Persistent Missions (Level 2):** Non-atomic, match-wide rules that remain active for the entire run and are maintained in dedicated runtime data structures. These include directional delivery constraints that restrict the agent to specific drop-off zones (`generate_delivery_constraint`); reward-multiplier rules conditioned on the parcel count or total carried score at delivery time (`generate_stack_constraint`); and tile-specific delivery bonuses or blocks (`generate_desire` with `go_delivery` or `avoid_delivery` sub-actions). The corresponding state is stored in dedicated maps and lists (`llmDeliveryBonusTiles`, `llmStackConstraints`, `llmBlockedDeliveryTiles`) that are consulted at every BDI step to bias or override autonomous utility scoring.
- **Coordination or Communication Missions (Level 3):** High-complexity tasks that require explicit peer-to-peer communication between the two agents (`multi_agent_command`). These cover four sub-commands: *rendezvous* at an explicit grid position within a configurable maximum distance; *wait_row* (red-light) to stop both agents on a row of specified parity; *resume* (green-light) to lift a wait hold; and *parcel_handoff*, where one agent picks up a parcel and meets the other at a BFS-computed meeting tile so the latter can deliver it for a bonus reward.

The core logic is managed by an orchestrator through the *processMessage* function, which executes the following pipeline:

1. **Mathematical Pre-processing:** The message is first passed to the *mathExtractor* (first LLM call). It identifies nested arithmetic operations, replaces them with unique placeholders (e.g., $X1, X2$), and returns a structured JSON object. The orchestrator then invokes a deterministic *calculate* tool to resolve these expressions and injects the actual results back into the text. This prevents LLM hallucinations and gracefully handles complex, nested calculations.
2. **Utterance Splitting:** The cleaned message is processed by the *splitter* component (second LLM call), which divides concatenated or complex queries into a list of independent sub-goals (e.g., splitting "go to tile 0,0 and tell me the square root of four" into two separate tasks).
3. **Action Classification:** Each sub-goal is evaluated by the *orchestrator* (third LLM call) to predict one of eight action labels: `calculate`, `get_current_time`, `get_my_position`, `common_knowledge`, `generate_desire`, `generate_delivery_constraint`, `generate_stack_constraint`, or `multi_agent_command`.
4. **Parameterized Dispatch:** Based on the predicted action, a dedicated extractor (additional LLM call) retrieves the structured parameters required by that action and populates the corresponding field of the `LLMUpdate` object. Direct-reply actions (`calculate`, `get_current_time`, `get_my_position`, `common_knowledge`) append a text answer to the response string. BDI-update actions populate typed fields within `LLMUpdate`: `goToTiles` and `blockedTiles` for movement goals and avoidances, `deliveryConstraints` for directional drop-off preferences, `deliveryBonusTiles`

and `blockedDeliveryTiles` for per-tile delivery modifiers, `stackConstraints` for reward-multiplier rules, and `multiAgentCommand` for the current coordination command.

The `generate_desire` action deserves special attention: its dedicated extractor produces one of four sub-actions — `go_to` (navigate to a tile for bonus points), `avoid` (block a tile entirely), `go_delivery` (register a delivery reward multiplier at a specific tile), or `avoid_delivery` (forbid delivery at a tile while still allowing traversal). This design cleanly separates movement goals from delivery-zone preferences without requiring separate orchestrator action labels.

The orchestrator aggregates the results, returning both a textual answer (to be replied via chat) and the populated `LLMUpdate` object to be injected into the agent’s state.

While this multi-step pipeline introduces a minor computational overhead due to consecutive LLM calls, it remains fully sustainable within the project’s scope. The challenge constraints are relatively relaxed, as requests occur sporadically and do not require real-time, millisecond-range execution; this makes architectural robustness and accuracy preferable over raw speed. Moreover, this modular design allows for individual performance evaluation of each component, enabling targeted adjustments at every step.

The **tools** implemented to support the agent’s actions are contained in a dedicated module that exposes the following functions:

- **calculate:** A secure mathematical expression evaluator. Instead of using a dangerous global *eval*, it parses the input string, validates it against a strict whitelist of allowed characters and syntax to prevent code injection, and executes the calculus using a safe subset of the native JavaScript *Math* object (supporting functions like *sqrt*, *pow*, *sin* and constants like π and *e*).
- **getCurrentTime:** Retrieves the local date and time for a given city. It dynamically imports the external library *city-timezones* to look up the correct timezone. The time formatting is handled natively via the JavaScript *Intl.DateTimeFormat* API.
- **getMyPosition:** A utility function that reads the agent’s current state and returns a formatted string with its coordinates (*x, y*).

All calculated answers and structured game-state updates are subsequently forwarded also to the *only BDI* agent, ensuring it possesses all the necessary data to execute the required actions and successfully claim the points for the completed request.

2.5 Utility Functions

The utility functions define the heuristics used by the agent to rank candidate desires. Each utility combines the expected reward of an action with its travel cost and possible risk factors.

2.5.1 Pick up utility

For a `go_pickup` desire targeting parcel *p* at position **q**, the utility is computed as the sum of two terms: the expected value of the parcels already carried by the agent and the expected value of the new parcel. The final score is then reduced when opponent agents are close to the pickup location:

$$u_{\text{pickup}}(p) = (V_{\text{carry}}(\mathbf{q}) + V_{\text{parcel}}(p, \mathbf{q})) \cdot P_{\text{risk}}(\mathbf{q}) \quad (1)$$

where

$$V_{\text{carry}}(\mathbf{q}) = R_{\text{carry}} - n \cdot \delta \cdot d(\mathbf{a}, \mathbf{q}) \quad (2)$$

is the estimated remaining value of the n parcels already carried (R_{carry}) when reaching the pickup position $\mathbf{q}$ from the current agent position $\mathbf{a}$, penalizing it based on the decay rate δ . This prevents the agent from continuously collecting parcels without ever delivering them, which is especially useful in scenarios characterized by a high abundance of parcels and a low density of competing agents. Then,

$$V_{\text{parcel}}(p, \mathbf{q}) = r_p - \delta \cdot d(\mathbf{q}, \mathbf{d}) \quad (3)$$

is the estimated net value of the new parcel, calculated by subtracting the expected decay incurred during the transit from its position $\mathbf{q}$ to the nearest delivery tile $\mathbf{d}$ from its base reward r_p . The final term, $P_{\text{risk}}(\mathbf{q})$, represents an environmental safety coefficient bounded between 0 and 1 that penalizes the target's utility based on the proximity of threats modeling the operational risk of the parcel being intercepted or stolen before the agent can reach the coordinates $\mathbf{q}$.

2.5.2 Delivery utility

For a `go_delivery` desire targeting delivery tile $\mathbf{d}$, the utility estimates the remaining value of the currently carried parcels after the trip:

$$u_{\text{delivery}}(\mathbf{d}) = (R_{\text{carry}} - n \cdot \delta \cdot d(\mathbf{a}, \mathbf{d})) \cdot P_{\text{risk}}(\mathbf{d}) \quad (4)$$

where R_{carry} is the current reward of the carried items, which decays at a rate of δ for each of the n parcels over the distance $d(\mathbf{a}, \mathbf{d})$ from the agent's position $\mathbf{a}$ to the delivery tile.

The penalty term $P_{\text{risk}}(\mathbf{d})$ handles stationary threats. The agent targets the three closest hazard-free delivery tiles ($P_{\text{risk}} = 1$). If all available delivery positions are blocked by stationary enemies, the agent falls back to these targets but scales their utility by $P_{\text{risk}} = 0.1$ to penalize the high-risk action.

2.5.3 Explore utility

For an `explore` desire targeting tile s , the utility rewards tiles that have not been visited recently, balanced against the distance required to reach them:

$$u_{\text{explore}}(s) = \frac{T_{\text{unvisited}}(s)}{d(\mathbf{a}, s) + 1} - C_{\text{travel}}(s) \quad (5)$$

where $T_{\text{unvisited}}(s)$ represents the time elapsed since the tile s was last visited, and $C_{\text{travel}}(s) = K_{\text{penalize}} \cdot \delta \cdot d(\mathbf{a}, s)$ defines the travel cost, heavily penalizing distant targets based on a tuning constant K_{penalize} and the map decay rate δ . This score is further adjusted by environmental multipliers to favor high-density parcel spawns while avoiding areas with high opponent pressure.

2.6 Main Strategies

Stuck and bounce recovery. The agent implements two deterministic recovery mechanisms to prevent deadlocks and repetitive, ineffective behaviors. First, if the pathfinder fails to compute a valid route for the same intention across `STUCK_THRESHOLD` consecutive reasoning cycles, the target intention is temporarily blacklisted for `INTENTION_BLOCK_MS`, forcing the agent to switch to an alternative candidate desire. Second, the agent monitors its recent coordinate history to detect cyclic oscillations, such as $A \rightarrow B \rightarrow A \rightarrow B$ patterns. When such a bounce loop is identified, the involved cells are temporarily blocked, the current path is discarded, and a new route is forced.

Collision avoidance. To ensure smooth navigation, the agent proactively predicts potential collisions by estimating the immediate future position of nearby opponents based on their last observed moving direction. If the agent’s next planned step coincides with a predicted opponent position, the current path is cleared and immediately recomputed to avoid a confrontation. Furthermore, opponents that remain stationary across multiple consecutive sensing updates are dynamically registered as static obstacles within the pathfinding grid, allowing the agent to route around them efficiently.

2.7 Communication Mechanisms

The multi-agent architecture follows a **master/slave** deployment model. One agent is designated as the *master* (`IS_MASTER=true`) and the other as the *slave*. The master agent is the sole point of entry for all natural-language instructions: it runs the full LLM pipeline and, immediately after each `processMessage` call, forwards the resulting `LLMUpdate` payload to the slave via a `LLM_UPDATE` peer message. Both agents then apply the same structured update to their respective BDI loops, ensuring synchronized belief state without requiring the slave to carry its own LLM client.

Communication relies on the Deliveroo.js peer-to-peer messaging layer (`socket.emitSay` / `socket.onMsg`). Partner discovery is automatic: the first non-admin incoming message is used to register the partner’s agent ID. Five dedicated message kinds support the coordination protocols:

- `LLM_UPDATE`: carries the full `LLMUpdate` payload from master to slave after each processed instruction.
- `RENDEZVOUS_ARRIVED`: broadcast by an agent that has entered the rendezvous radius, signalling that it is in position. Upon receipt, an agent already within range immediately completes the mission.
- `HANDOFF_SLAVE_POS`: sent by the slave (deliverer) immediately upon receiving a `parcel_handoff` command, reporting its current coordinates so the master (picker) can compute the optimal meeting tile.
- `HANDOFF_APPROACH`: sent by the picker once it has acquired a parcel, notifying the deliverer of the picker’s current position so the deliverer can start converging.
- `HANDOFF_DROPPED`: sent by the picker after it has dropped the parcel at the meeting tile, including the parcel list and drop coordinates, so the deliverer can navigate directly to the tile for pickup.

Parcel Handoff Protocol. When a `parcel_handoff` command is received, the master assumes the *picker* role and the slave assumes the *deliverer* role. The protocol proceeds in three phases:

1. **Meeting-point negotiation.** The slave immediately reports its position via `HANDOFF_SLAVE_POS`. The master waits up to a configurable timeout; if the report arrives in time, it runs a BFS-based meeting-point algorithm (`computeMeetingPoint`) that finds the walkable tile minimizing the sum of BFS distances from both agents, preferring tiles that are not trivially reachable in zero steps by either agent. If the slave position is not received within the timeout, the master aborts the handoff and reverts to normal delivery.
2. **Acquisition and approach.** The picker acquires a target parcel via its normal BDI loop (highest-utility `go_pickup` desire). Once a parcel is in hand, it transitions to the `handoff_approach` phase, navigates toward the computed meeting tile, and sends `HANDOFF_APPROACH` to the deliverer. The deliverer navigates to the same meeting tile. While in any handoff phase, both agents suppress normal delivery intentions to avoid premature drop-offs.
3. **Drop and pickup.** When the picker reaches the meeting tile (Manhattan distance ≤ 1), it drops the parcel via `emitPutdown` and broadcasts `HANDOFF_DROPPED` with the drop coordinates and parcel list. The deliverer’s belief state is pre-populated with the dropped parcel to ensure a non-empty pickup queue on arrival. The deliverer then picks up the parcel and delivers it for the bonus reward. A per-agent timeout at the meeting tile ensures the protocol degrades gracefully if either agent becomes stuck: the picker drops on timeout, and the deliverer abandons the wait after a separate timer expires.

3 Experimental Setup

3.1 Scenarios

The agent architecture was evaluated across two main configurations based on the challenge scenarios, specifically designed to test planning efficiency and multi-agent coordination:

- **Single-Agent Benchmarking:** Evaluated on the **first challenge** scenario to isolate and compare planning performance. This setup contrasts the default configuration using Breadth-First Search (BFS) against the proposed PDDL planner.
- **Multi-Agent Coordination:** Evaluated on the **second challenge** scenario to analyze team dynamics. This setup compares a baseline deployment operating without communication against a cooperative team of two agents, where both utilize PDDL planning but their architectures differ: one agent operates solely via BDI, while the other is enhanced with BDI and LLM integration.

To measure the performance and efficiency of each configuration, we consider the **net score** ($R_{\text{net}} = \text{reward} - \text{penalties}$) accumulated by the agents at the end of each map, following a 5-minute run per map.

3.2 LLM Component Evaluation Setup

The LLM component is evaluated in isolation via a dedicated test suite (`tests/llm_tests.ts`), with test cases stored as JSON files under `tests/data/`. Rather than evaluating the system as a monolithic block, the suite isolates individual pipeline stages across **315 total test cases** spanning 11 categories to enable a fine-grained diagnosis of failure modes.

The dataset covers every stage of the LLM pipeline. Deterministic categories include the tool handlers (`tools`, 53 cases) and the action label classifier (`extractAction`, 25 cases). LLM-backed categories target each processing step individually: math expression isolation and placeholder resolution (`planTasks`, 30 cases; `extractMathExpression`, 25 cases), request decomposition (`splitMessage`, 35 cases), geographic entity extraction (`extractCityName`, 25 cases), structured JSON constraint parsing for delivery directions (`extractDeliveryConstraint`, 25 cases) and parcel-stack rules (`extractStackConstraint`, 8 cases), tile desire generation including sub-action discrimination (`generateDesire`, 23 cases), orchestrator action classification across all eight action types (`decideNextAction`, 44 cases), and multi-agent coordination command extraction covering all four sub-commands and both explicit and implicit stop/resume signals (`extractMultiAgentCommand`, 22 cases). Ground-truth verification uses exact string matching for classification labels and specialized JSON structural validation for complex constraint objects.

4 Results

Table 1 reports the results on the single agent scenario comparing the **net score** of BFS planning with PDDL using A^* to find the shortest path to reach a tile.

Table 1: Single-agent performance BFS vs PDDL with average net scores.

Map	BFS	PDDL
26c1_1	1416	1310
26c1_2	1630	1582
26c1_3	894	959
26c1_4	1463	1443
26c1_5	681	782
26c1_6	2063	2364
26c1_7	1766	1794
26c1_8	1949	2044
Average	1482.75	1534.75

The performance variations depend strictly on the structural characteristics of each map. BFS achieves higher net scores on maps that are less constrained and well-connected, where sub-millisecond execution allows the agent to react instantly to fast environmental updates. Conversely, on more complex maps (e.g., 26c1_3 and 26c1_5 through 26c1_8), the strategic value of optimal action sequences computed by the PDDL planner successfully offsets its call overhead. This structural advantage is reflected in the overall performance, with the PDDL pipeline achieving a higher average net score of 1534.75 compared to the 1482.75 baseline.

TODO: Add results challenge 2

4.1 LLM Components Evaluation Results

Table 2 reports a component-level evaluation of the LLM pipeline, measuring the accuracy and average latency of each processing stage across all 315 test cases. Given the focus on architectural validation, the metrics were collected on a controlled test set designed to cover the full range of prompt complexity for each category. While the high overall accuracy (98.4%) demonstrates the effectiveness of the targeted few-shot prompts, these figures serve primarily to contextualize execution latency and structural robustness rather than to claim absolute generalization over larger, unconstrained datasets.

Table 2: Component-level evaluation of the LLM pipeline (315 total test cases).

Category	Total	Passed	Accuracy	Avg. latency (ms)
<code>extractAction</code>	25	25	100.0%	0
<code>tools</code>	53	52	98.1%	2
<code>splitMessage</code>	35	34	97.1%	867
<code>extractCityName</code>	25	25	100.0%	170
<code>extractMathExpression</code>	25	24	96.0%	278
<code>planTasks</code>	30	28	93.3%	1206
<code>extractDeliveryConstraint</code>	25	25	100.0%	513
<code>generateDesire</code>	23	23	100.0%	1401
<code>extractStackConstraint</code>	8	8	100.0%	1549
<code>extractMultiAgentCommand</code>	22	22	100.0%	509
<code>decideNextAction</code>	44	44	100.0%	203
Total	315	310	98.4%	—

The two deterministic categories (`extractAction` and `tools`) execute in near-zero latency and achieve near-perfect accuracy, validating the correctness of the whitelist-based action classifier and the safe expression evaluator. Among the LLM-backed stages, `planTasks` (93.3%) and `extractMathExpression` (96.0%) show the highest failure rates, concentrated on edge cases involving nested or multi-step expressions. All remaining categories — including the newly added `extractStackConstraint`, `generateDesire`, and `extractMultiAgentCommand` — achieve 100% accuracy, confirming that the targeted few-shot prompts generalize well to the full range of Level 2 and Level 3 mission types. Latency is highest for categories that require complex structured JSON extraction (`extractStackConstraint`: 1549 ms, `generateDesire`: 1401 ms), consistent with the greater reasoning load imposed by those prompts.

5 Limitations

- **LLM Latency Overhead:** The sequential multi-call pipeline introduces a cumulative latency. While acceptable for this challenge, it is unfeasible for real-time environments requiring millisecond-range responses.
- **Lack of Competitive Evaluation:** Benchmarks were conducted in non-competitive scenarios without active adversarial agents. Testing against dynamic opponents

is required to fully validate the resilience of the utility functions and strategic decision-making.

- **Statistical Significance:** Because the results were collected from a single execution run per map, they lack a solid statistical baseline. Multiple runs are necessary to account for environmental stochasticity and evaluate performance consistency.

6 Conclusion